

UNIVERSIDADE DA CORUÑA

Programación de Sistemas

2.6 Advanced UI

Carlos Vázquez Regueiro
carlos.vazquez.regueiro@udc.es

Índice

1. **Toast y Snackbar**
2. Notificaciones
3. Añadir botones a una notificación
4. Tipos de menú y cómo crearlos
5. Menú de opciones
6. Menú contextual
7. Permisos
8. Foreground Services

Toast (I)

- Forma simple de mostrar un mensaje en pantalla
- Se oculta automáticamente pasado un tiempo
- Solo ocupa la cantidad de espacio necesario para el mensaje
- La actividad en curso permanece visible y admite la interacción
- Normalmente se utiliza para proporcionar feedback al usuario
  - P. ej.: Enviando mensaje…; Mensaje enviado
- Tiene un layout por defecto pero se le puede adaptar al estilo de la app
  - Método *setView();*

Toast (y II)

- Declaración:

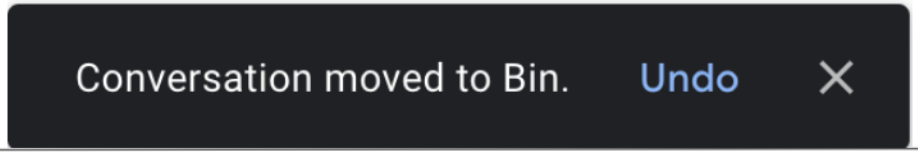
```
Context context = getApplicationContext();
CharSequence text = "Hello toast!";
int duration = Toast.LENGTH_SHORT; // Toast.LENGTH_LONG
Toast toast = Toast.makeText(context, text, duration);
toast.setGravity(Gravity.TOP|Gravity.LEFT, 0, 0);
toast.show();
```

- Importante acordarse de llamar al método **show()**
- Normalmente se llama ya directamente en una sola línea:

```
Toast.makeText(this, "Hello toast!", Toast.LENGTH_SHORT).show();
```

Snackbar (I)

- Similar al Toast pero con más opciones de personalización
- Se muestran encima de los otros elementos del layout
- Sólo se puede mostrar uno de cada vez
- Desaparecen automáticamente después de timeout o por interacción del usuario
- Los Snackbars pueden tener acciones asociados
  - P. ej. Gmail (tanto app como web) si se borra un mensaje lanza un aviso y contempla la opción de deshacer la operación



Snackbars (II)

- Declaración:

```
Snackbar mySnackbar = Snackbar.make(view, stringId, duration);
mySnackbar.show();
```

- **view**: es la vista a la que se adjunta el Snackbar
  - El método busca la jerarquía hasta llegar a un *CoordinatorLayout*
    - o a la vista de contenido de la decoración de la ventana (es más sencillo pasar el *CoordinatorLayout* que incluye tu contenido.stringId
- **stringId:** id del recurso del mensaje o el mensaje en sí
- **duration:** Tiempo que se muestra el mensaje (LENGTH_SHORT o LENGTH_LONG)

Snackbars (y III)

- A los Snackbars se le pueden añadir acciones

```
Snackbar mySnackbar = Snackbar.make(binding.main, stringId, duration);
mySnackbar.setAction(R.string.undo_string, new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // Code to undo the user's last action
    }
});
mySnackbar.show();
```

Índice

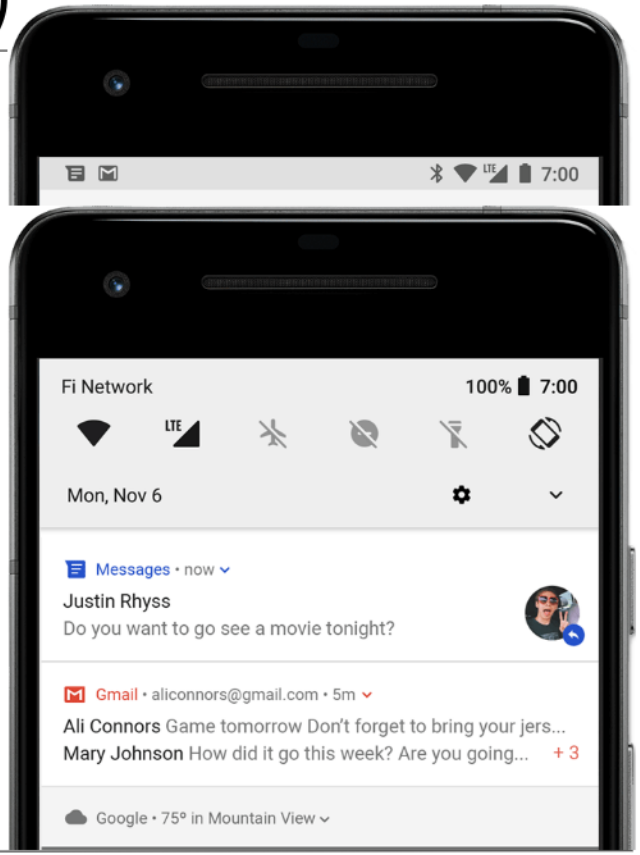
1. Toast y Snackbar
2. **Notificaciones**
3. Añadir botones a una notificación
4. Tipos de menú y cómo crearlos
5. Menú de opciones
6. Menú contextual
7. Permisos
8. Foreground Services

Notificaciones (I)

- Mensaje que se muestra fuera de la IU de la app para proporcionar al usuario recordatorios, mensajes de otras personas o información específica de la app
- Son clickables, y pueden:
  - Abrir la app
  - Realizar una determinada acción
- Se muestran en diferentes ubicaciones y formatos
  - Icono en la barra de estado
  - Una entrada más detallada en el panel de notificaciones
  - Una insignia en el icono de la app

Notificaciones (y II)

- Cuando llega una notificación, primero aparece como un icono en la barra de estado
- Los usuarios pueden deslizar hacia abajo la barra de estado para abrir el panel de notificaciones, donde pueden ver más detalles y realizar acciones desde la notificación
- Desde Android 5 se añadieron, nuevos permisos, prioridades, notificaciones emergentes, …
- Guías de estilo propias y muy detalladas



Creación de una notificación básica

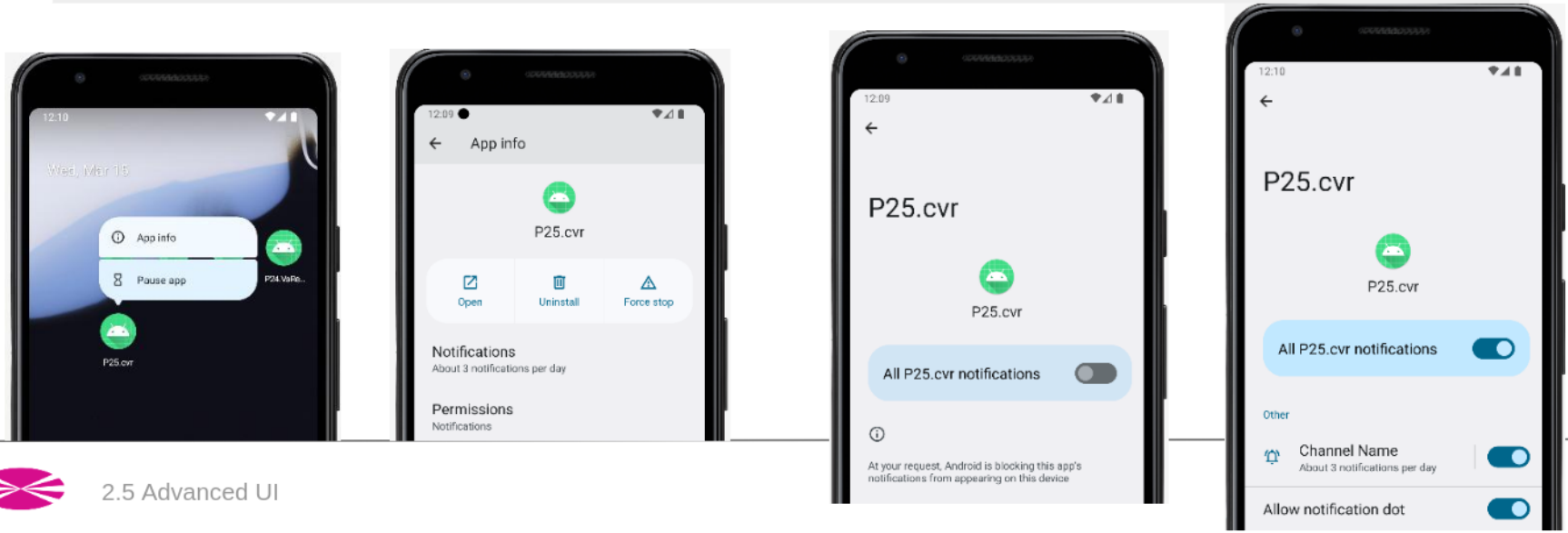
```
NotificationCompat.Builder builder = new NotificationCompat.Builder(this, CHANNEL_ID)
        .setSmallIcon(R.mipmap.ic_launcher_round)
        .setContentTitle("1a notificacion")
        .setContentText("contenido 1a notificación")
        .setPriority(NotificationCompat.PRIORITY_DEFAULT);
```

- El texto se trunca para que quepa en una línea
- Para crear notificaciones más grandes, ver la documentación
- CHANNEL_ID: id del canal (necesario a partir de Android 8.0)
- Establecemos icono (único contenido necesario visible para el usuario), título, contenido y prioridad (cuán de intrusiva debería ser la notificación (Android 7.1))
  - Android 8.0 y posteriores se debe establecer la importancia del canal

Notificación: permisos

- Desde API 33, la aplicación debe solicitar el permiso para enviar notificaciones
- El permiso es revocable por el usuario (debería ser comprobado) ver documentación
  - Hay que dar permiso a nuestra aplicación para que pueda lanzar notificaciones

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
```



Creación de un canal y su importancia (I)

```
private void createNotificationChannel() {

    // Create the NotificationChannel, but only on API 26+ because
    // the NotificationChannel class is new and not in the support library
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        CharSequence name = getString(R.string.channel_name);
        String description = getString(R.string.channel_description);
        int importance = NotificationManager.IMPORTANCE_DEFAULT;

        NotificationChannel channel = new NotificationChannel(CHANNEL_ID,
                                                name, importance);
        channel.setDescription(description);
        // Register the channel with the system; you can't change the importance
        // or other notification behaviors after this
        NotificationManager notifManager = getSystemService(NotificationManager.class);
        notifManager.createNotificationChannel(channel);
    }
}
```

Creación de un canal y su importancia (y II)

- Se debería ejecutar este código tan pronto como se inicie la app
  - Se considera que realizar llamadas repetidas es seguro porque, cuando se crea un canal de notificación, no se lleva a cabo ninguna operación
- En **importance** hay que usar una de las constantes de la clase NotificationManager
  - Este parámetro determina cómo interrumpir al usuario
  - También se debe definir la prioridad con setPriority() para admitir Android 7.1
- El sistema no garantiza que el comportamiento de la alerta es el que defines
  - Dependerá de muchas variables

Establecer la acción de toque de la notificación (I)

- Cada notificación debe responder a un toque
  - Casi siempre es para abrir la actividad en la app que se corresponda con la notificación
  - Hay que especificar un intent de contenido (PendingIntent) y pasarlo a setContentIntent()
- El toque puede ocultar la notificación
  - Se configura con *autoCancel()*

Establecer la acción de toque de la notificación (y II)

```
Intent intent = new Intent(this, MainActivity.class);
intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
PendingIntent pendingIntent = PendingIntent.getActivity(this,
          requestCode,        // cualquier valor entero: distingue intents iguales
          intent,             // el intent que se quiere lanzar
          PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE );

NotificationCompat.Builder builder =
     new NotificationCompat.Builder(this, CHANNEL_ID)
          .setSmallIcon(R.mipmap.ic_launcher_round)
          .setContentTitle("My intent notification")
          .setContentText("Description of the intent")
          .setPriority(NotificationCompat.PRIORITY_DEFAULT)
          // Set the intent that will fire when the user taps the notification
          .setContentIntent(pendingIntent)
          .setAutoCancel(true);
```

Mostrar la notificación

- NotificationManagerCompat.notify(id, notification)
  - ID único para la notificación
  - NotificationCompat.Builder.build()

```
NotificationManagerCompat notificationManager =
                                        NotificationManagerCompat.from(this);

// notificationId is a unique int for each notification that you must define
notificationManager.notify(notificationId, builder.build());
```

- El ID de la notificación se necesita para actualizar o quitar la notificación

Índice

1. Toast y Snackbar
2. Notificaciones
3. **Añadir botones a una notificación**
4. Tipos de menú y cómo crearlos
5. Menú de opciones
6. Menú contextual
7. Permisos
8. Foreground Services

Añadir botones a la notificación (I)

- Se pueden añadir hasta tres botones de acción
  - Permitan al usuario responder de manera rápida
    - Posponer un recordatorio, incluso responder un mensaje de texto, ...
  - No deben duplicar la acción realizada al presionar la notificación
- addAction(): se le pasa un elemento PendingIntent
  - Similar a la acción de toque predeterminada de la notificación
  - Puede llevar a cabo una variedad de otras funciones
- Se pueden realizar acciones complejas: controles de un reproductor multimedia

Añadir botones a la notificación (y II)

```
Intent snIntent = new Intent(this, MyBroadcastReceiver.class);
snIntent.setAction(ACTION_SNOOZE);
snIntent.putExtra(EXTRA_NOTIFICATION_ID, 0);
PendingIntent snoozePendingIntent =
       PendingIntent.getBroadcast(this, 0, snIntent, PendingIntent.FLAG_IMMUTABLE);

NotificationCompat.Builder builder =
     new NotificationCompat.Builder(this, CHANNEL_ID)
       .setSmallIcon(R.drawable.notification_icon)
       .setContentTitle("My notification")
       .setContentText("Hello World!")
       .setPriority(NotificationCompat.PRIORITY_DEFAULT)
       .setContentIntent(pendingIntent)

       .addAction(R.drawable.ic_snooze, getString(R.string.snooze),
              snoozePendingIntent);
```

Añadir barra de progreso (I)

- Las notificaciones pueden incluir una barra de progreso animado
  - Muestra el estado de alguna operación
  - P. ej.: Tiempo de descarga



- setProgress(max, progress, indeterminate).
  - max: valor que indica que la operación está completa
  - progress: qué parte está completa
  - indeterminate: si podemos saber cuando acaba (false)

Añadir barra de progreso (y II)

```
NotificationManagerCompat notificationManager = NotificationManagerCompat.from(this);
NotificationCompat.Builder builder = new NotificationCompat.Builder(this, CHANNEL_ID);
builder.setContentTitle("Picture Download").setContentText("Download in progress")
       .setSmallIcon(R.mipmap.ic_launcher)
       .setPriority(NotificationCompat.PRIORITY_LOW);

int PROGRESS_MAX = 100;
int PROGRESS_CURRENT = 0; // Issue the initial notification with zero progress
builder.setProgress(PROGRESS_MAX, PROGRESS_CURRENT, false);
notificationManager.notify(notificationId, builder.build());
```

```
// Do the job here that tracks the progress.
// To show progress, update PROGRESS_CURRENT and update the notification with:
// builder.setProgress(PROGRESS_MAX, PROGRESS_CURRENT, false);
// notificationManager.notify(notificationId, builder.build());
// When done, update the notification one more time to remove the progress bar
builder.setContentText("Download complete").setProgress(0,0,false);
notificationManager.notify(notificationId, builder.build());
```

Índice

1. Toast y Snackbar
2. Notificaciones
3. Añadir botones a una notificación
4. **Tipos de menú y cómo crearlos**
5. Menú de opciones
6. Menú contextual
7. Permisos
8. Foreground Services

Tipos de menú

- Menú de opciones: colección principal de elementos de menú de una actividad
  - Acciones que tienen un impacto global en la app
- Menú contextual: menú flotante que aparece en un click largo en un elemento
  - Sólo afectan a los elementos seccionados o en el marco contextual
- Menú emergente: lista de elementos en una lista anclada a la vista que lo invocó
  - Son acciones extendidas y NO deben afectar al contenido correspondiente

Crear un menú (I)

- Programáticamente (se utiliza en pocas ocasiones):
  - Se crea el menú desde la actividad
  - Es fácil de cambiar dinámicamente
- XML: más genérico y “reutilizable”
  - Definir un recurso en res/menu
  - Crear (inflate) el menú en la actividad
  - Aunque se puede adaptar dinámicamente después por programa

Crear un menú (II)

- Fichero xml con 3 tipos de elementos:
  - <menu>: raíz de la jerarquía (puede ser anidada: con submenús)
  - <item>: cada elemento de menú “visible” (con diferentes propiedades)
  - <group>: para agrupar elementos (items)
- Cada elemento requiere al menos 3 elementos:
  - android:id para identificar unívocamente cada elemento
  - android:title referencia a la cadena que será el título
  - android:icon referencia al elemento que será el icono del elemento
- app:showAsAction indica cuándo y cómo debe aparecer en la barra de app
  - ["ifRoom" | "never" | "withText" | "always" | "collapseActionView"]

Crear un menú (y III)

```
<menu xmlns:app="http://schemas.android.com/apk/res-auto"
   xmlns:android="http://schemas.android.com/apk/res/android">

   <item android:id="@+id/menu_add_data"
       android:icon="@android:drawable/ic_input_add"
       android:title="@string/add_data"
       app:showAsAction="ifRoom" />

   <item android:id="@+id/menu_help"
       android:icon="@android:drawable/ic_input_add"
       android:title="@string/help" />

   <item android:id="@+id/menu_remove_data"
       android:title="@string/remove_data" />

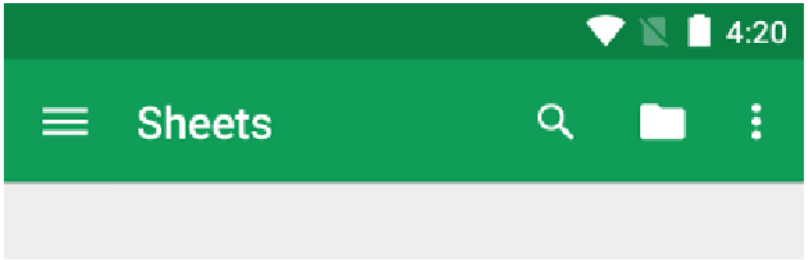
</menu>
```

Índice

1. Toast y Snackbar
2. Notificaciones
3. Añadir botones a una notificación
4. Tipos de menú y cómo crearlos
5. **Menú de opciones**
6. Menú contextual
7. Permisos
8. Foreground Services

Menú de opciones

- Creación:
  - Android 2.3.x (API 10): se crea al pulsar el botón “Menú”
  - > Android 3.0 (API 11): se crea al iniciar la barra de la app (nivel actividad)
    - Los fragmentos pueden añadir/eliminar sus propios elementos a la barra (que gestionará él mismo)



Crear menú de opciones

- Sobrecargar método ***onCreateOptionsMenu()*** en cada actividad:
  - Llamar al recurso xml correspondiente
  - No olvidar devolver ***true***

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.main_menu, menu);
    return true;
}
```

Cambiar menú de opciones

- Android < 2.3.x (API 10): se crea cada vez que se abre el menú
- Android > 3.0 (API 11): el menú de opciones siempre está abierto
  - Primero llamar a ***invalidateOptionsMenu()*** => sistema crea nuevo menú
  - Se pueden añadir nuevos elemento con el método: ***add()***
  - Se pueden recuperar elementos con el método: ***findItem()***
- Se pueden modificar cada una de las propiedades del elemento:
  - Visible: true|false
  - Enable: true|false
  - Checkable: true|false

Eventos menú de opciones

- Sobrecargar método ***onOptionsItemSelected()*** e identificar el id con ***getItemId()***:
  - No olvidar devolver ***true*** (consumir evento) y llamar al método del padre
  - Primero la actividad y después todos sus fragmentos (hasta consumir evento)

```
@Override
public boolean onOptionsItemSelected(@NonNull MenuItem item) {
    if (item.getItemId() == R.id.menu_add_data) {
        Toast.makeText(this, getString(R.string.menu_add), Toast.LENGTH_SHORT).show();
    } else if (item.getItemId() == R.id.menu_help) {
        Toast.makeText(this, getString(R.string.menu_help), Toast.LENGTH_SHORT).show();
    } else if (item.getItemId() == R.id.menu_remove_data) {
        Toast.makeText(this, getString(R.string.menu_remove), Toast.LENGTH_SHORT).show();
    } else {
        return super.onOptionsItemSelected(item);
    }
    return true;
}
```

Índice

1. Toast y Snackbar
2. Notificaciones
3. Añadir botones a una notificación
4. Tipos de menú y cómo crearlos
5. Menú de opciones
6. **Menú contextual**
7. Permisos
8. Foreground Services

Menú contextual

- Se puede proporcionar un menú contextual a cualquier vista
  - Colecciones (ListView, GridList, ...)
- Menú contextual flotante
  - Similar a cuadro de diálogo
  - Sólo 1 elemento de cada vez
- Modo de acción contextual:
  - Barra de acciones (ActionMode)
  - Se puede aplicar a varios elementos



Menú contextual flotante (I)

1. Registra la *View* con la que se debe asociar el menú con ***registerForContextMenu()***

```
textView = findViewById(R.id.hello_tv);
registerForContextMenu(textView);
```

   - Si la vista es una colección se asigna el menú a todos sus elementos (se debe definir un adaptador o usar uno del sistema). OJO: *ListView* está obsoleto

```
listView = findViewById(R.id.listView);

String[] data = {"one", "two", "three", "four", "five", "six", "seven"};
ArrayList<String> dataList = new ArrayList<String>(Arrays.asList(data)); //dynamic list
ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
                                android.R.layout.simple_list_item_1, dataList);
listView.setAdapter(adapter);

registerForContextMenu(listView);
```

Menú contextual flotante (II)

2. Sobrecarga el método ***onCreateContextMenu()*** en la actividad o fragmento
   - Se suele utilizar un recurso XML (aunque se podría hacer por programa)
   - Se puede personalizar según la vista y el contexto

```
@Override
public void onCreateContextMenu(ContextMenu menu, View v,
                                ContextMenu.ContextMenuInfo menuInfo) {
   super.onCreateContextMenu(menu, v, menuInfo);
   MenuInflater inflater = getMenuInflater();
   inflater.inflate(R.menu.contextual_menu, menu);
}
```

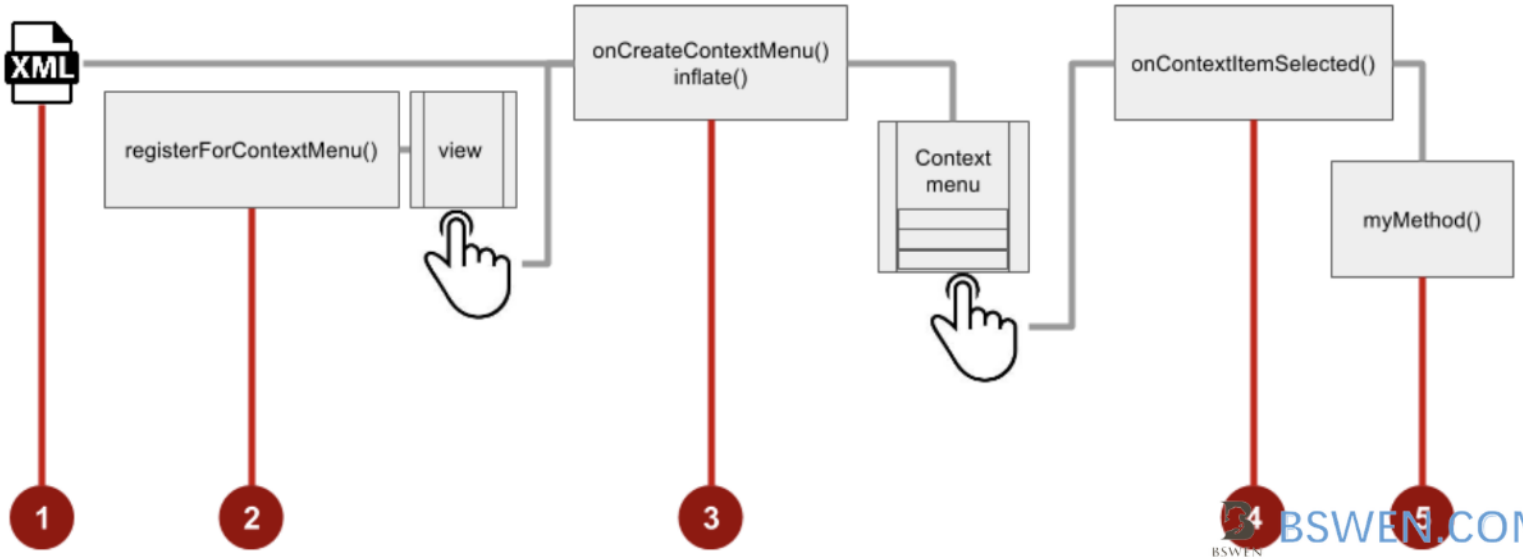
Menú contextual flotante (y III)

3. Para gestionar los eventos, sobrecarga el método ***onContextItemSelected()***:
    a. Ojo: *info* sólo existe en las colecciones (de lo contrario es null)

```
@Override
public boolean onContextItemSelected(@NonNull MenuItem item) {
   AdapterView.AdapterContextMenuInfo info =
               (AdapterView.AdapterContextMenuInfo) item.getMenuInfo();
   if (item.getItemId() == R.id.menu_contxt_delete) {
           dataList.remove(info.position);        // para el ejemplo del ListView
           adapter.notifyDataSetChanged();        // para el ejemplo del ListView
   } else if (item.getItemId() == R.id.menu_contxt_share) {
       Log.d("_TAG", "menu: "+ item.getTitle().toString() +" item: "+ info.id);
   } else  return super.onContextItemSelected(item);
   } return true;
}
```

Menú contextual flotante (y III)

3. Resumen



BSWEN.COM

Menú contextual con Recycler View (I)

1. Se registra el constructor del menú con el listener del recycler view

```
mAdapter.setClickListener(new ArticlesAdapter.OnItemClickListener() {
    @Override
    public void onClick(View view, int position) {
            openOptionMenu(view, position);
    }
});
```

Menú contextual con Recycler View (y II)

2. Se construye el menú específico para cada item y su gestión

```
public void openOptionMenu(View v,final int position){
    PopupMenu popup = new PopupMenu(v.getContext(), v);
    popup.getMenuInflater().inflate(R.menu.contextual_menu, popup.getMenu());
    popup.setOnMenuItemClickListener(new PopupMenu.OnMenuItemClickListener() {
        @Override
        public boolean onMenuItemClick(MenuItem item) {
            Toast.makeText(getBaseContext(),
                           "Action : " + item.getTitle() + " position " + position,
                           Toast.LENGTH_SHORT).show();
            // se puede usar: switch(item.getItemId()) { case R.id.menú_edit: … }
            return true;
        }
    });
    popup.show();
}
```

Índice

1. Toast y Snackbar
2. Notificaciones
3. Añadir botones a una notificación
4. Tipos de menú y cómo crearlos
5. Menú de opciones
6. Menú contextual
7. **Permisos**
8. Foreground Services

Permisos: intro

- Los permisos ayudan a mantener la privacidad del usuario y protegen:
  - **Datos restringidos** (p.e. estado del sistema o info personal usuario)
  - **Acciones restringidas** (p.e. conectarse a un dispositivo o grabar audio)

- Tipos de permisos:
  - **Normales**: suponen un riesgo mínimo y se dan en el momento de la instalación
  - **Firma**: comparten certificado de firma con el Sistema Operativo (fabricante)
  - **Peligrosos**: deben comprobarse en tiempo de ejecución (el usuario puede revocarlos)
  - **Especiales**: solo pueden estar definidos por la plataforma o el fabricante

Permisos: buenas prácticas

- Prácticas recomendadas para usar los permisos de forma efectiva:
  - **Solicita una cantidad mínima de permisos** (solo los imprescindibles)
  - **Asocia permisos de ejecución con acciones específicas** (comprobar ***siempre antes***, justo en el momento, de usar la función protegida)
  - Considera las **dependencias** de tu app (revisa las bibliotecas que empleas)
  - **Sé transparente** (explica siempre al usuario por qué necesitas cada permiso)

Declaración de permisos y características

- Ambos se declaran en el **Manifiesto**: los permisos son **obligatorios**, las características son **opcionales**

```
<manifest ...>

    <uses-permission android:name="android.permission.CAMERA"/>
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
    <application ...>
        ...
    </application>

    <uses-feature android:name="android.hardware.camera"
                  android:required="false" />
</manifest>
```

Características: comprobación

- Para determinar la disponibilidad de ciertas características hardware del sistema:

```
// Check whether your app is running on a device that has a front-facing camera
if (getApplicationContext().getPackageManager().hasSystemFeature(
        PackageManager.FEATURE_CAMERA_FRONT)) {
    // Continue with the part of your app's workflow that requires a
    // front-facing camera
} else {
    // Gracefully degrade your app experience
}
```

Permisos: comprobación (I)

- Flujo de trabajo asociado a la comprobación de permisos:

Permisos: comprobación (II)

1. Es necesario comprobar los permisos en tiempo de ejecución (antes de usar):

```
if (ContextCompat.checkSelfPermission(CONTEXT, Manifest.permission.REQUESTED_PERMISSION)
                    == PackageManager.PERMISSION_GRANTED) {
    // You can use the API that requires the permission
    performAction(...);
} else if (ActivityCompat.shouldShowRequestPermissionRationale(
        this, android.Manifest.permission.REQUESTED_PERMISSION)) {
    // Explain to the user why your app requires this permission for a specific feature
    // to behave as expected, and what features are disabled if it's declined.
    // Include a "cancel" or "no thanks" button that lets the user continue
    // using your app without granting the permission
    showInContextUI(...); // you also can use ➔ requestPermissionLauncher.launch()
} else {
    // You can directly ask for the permission.
    // The registered ActivityResultCallback gets the result of this request
    requestPermissionLauncher.launch(android.Manifest.permission.REQUESTED_PERMISSION);
}
```

Permisos: comprobación (y III)

2. Para gestionar las resupuestas del sistema necesitamos definir un *Launcher*:

```
private ActiviyResultLauncher<String> requestPermissionLauncher =
     registerForActivityResult(new RequestPermission(), isGranted -> {
          if (isGranted) {
              // Permission is granted. Continue the action or workflow in your app
              performAction(...);
          } else {
              // Explain to the user that the feature is unavailable because the
              // feature requires a permission that the user has denied. At the
              // same time, respect the user's decision. Don't link to system
              // settings in an effort to convince the user to change their
              // decision
          }
  });
```

Índice

1. Toast y Snackbar
2. Notificaciones
3. Añadir botones a una notificación
4. Tipos de menú y cómo crearlos
5. Menú de opciones
6. Menú contextual
7. Permisos
8. **Foreground Services**

Foreground Services: intro

- Foreground service es un **servicio** (se ejecuta en segundo plano) que tiene asociado una **notificación** para avisar al usuario que está activo y consumiendo recursos
- Desde Android 9 es obligatorio pedir **permiso** FOREGROUND_SERVICE en el Manifest

```
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
```

- También se debe pedir **permiso específico** según tipo (ver documentación):

```
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_CONNECTED_DEVICE"/>
```

- Si no se declara alguno de los 2 permisos necesarios, habrá una excepción:

```
java.lang.SecurityException:
    Permission Denial: startForeground from pid=8589, uid=10623
    requires android.permission.FOREGROUND_SERVICE
```

```
java.lang.SecurityException:
    Starting FGS with type mediaPlayback targetSDK=34
    requires permissions:
    all of the permissions allOf=true
    [android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK]
```

Foreground Services: tipos

- Desde Android 14 es **obligatorio** especificar a qué **tipo** pertenecen (Manifest):
  - camera: acceso a la cámara desde segundo plano
  - connectedDevice: acceso a dispositivos externos (Bluetooth, etc)
  - dataSync: para upload y download => será deprecated, usar otras alternativas
  - health: para fitness
  - location: cuando se requiere usar la localización
  - mediaPlayback o mediaProjection: audio-vídeo contínuo o en disp. externo
  - microphone: acceso al micrófono en segundo plano
  - phoneCall: llamadas
  - remoteMessaging: transferir mensajes de texto entre varios dispositivos
  - shortService: ejecución de tareas críticas no interrumpibles (máx. 3 minutos)
  - specialUse: cualquier otro uso (se deberá justificar bien antes subir a tienda)
  - systemExempted: reservado para aplicaciones del sistema

Foreground Services: declaración

- El foreground service debe estar declarado en el **Manifest** con su(s) tipo(s)

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android" ...>
    <service
        android:name=".MyForegroundService"
        android:foregroundServiceType="camera|location|microphone"
        android:exported="false">
    </service>
</manifest>
```

- Se lanza 1o el servicio y después se **promociona** con ***startForeground*** (antes de 10s):

```
ServiceCompat.startForeground(
    this,  // servicio
    id,    // no puede ser 0
    notification,  // no olvidar crear antes el canal (como en cualquier notificación)
    // p.e. → new NotificationCompat.Builder(this, "CHANNEL_ID").build()
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) {
        ServiceInfo.FOREGROUND_SERVICE_TYPE_MEDIA_PLAYBACK
    } else {   0   }
);
```



Foreground Services: uso

- El servicio inicial puede ser un local o enlazado
  - Si es enlazado se podrá interaccionar con él a través del IB*ind* que devuelve
- Si requiere permisos, debe comprobarlos siempre antes de usarlos
- Si dentro del servicio se llama a ***stopForeground(int)*** dejará de ser Foreground service, aunque seguirá como servicio y **continuará** su ejecución
- Un foreground service se puede parar a si mismo con *stopSelf()*

Referencias

- https://developer.android.com/guide/topics/ui/notifiers/toasts
- https://developer.android.com/training/snackbar/showing
- https://developer.android.com/training/notify-user/build-notification
- https://developer.android.com/training/notify-user/expanded
- https://developer.android.com/guide/topics/ui/menus
- https://developer.android.com/guide/topics/resources/menu-resource
- https://developer.android.com/guide/topics/permissions/overview
- https://developer.android.com/develop/background-work/services/fgs
- https://developer.android.com/develop/background-work/services/fgs/service-types
- https://medium.com/@domen.lanisnik/guide-to-foreground-services-on-android-9d0127dc8f9a